

LECTURE 23

Views and Materialized Views

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What We'll Cover

- What a view is: a virtual table with no stored data of its own
- Creating views with `CREATE VIEW ... AS SELECT ...`
- Querying a view exactly like a base table
- Which views are updatable, and which fundamentally cannot be
- Advantages and disadvantages of ordinary views
- Materialized views: what “materialized” actually changes
- Creating, refreshing, and choosing materialized views

Views: a Virtual Table

- A view's definition is a stored query — it has no rows of its own on disk
- Every query against a view re-runs the underlying query against current base-table data
- Nothing is duplicated: the view is a lens onto the base tables, not a copy
- A branch manager should only see staff at their own branch, not the whole company

Staff (**staffNo**, **name**, **position**, **salary**, **branchNo**)

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007

Creating Views

Filter rows for one branch

```
CREATE VIEW Branch003Staff AS
  SELECT staffNo, name, position, salary
  FROM Staff
  WHERE branchNo = 'B003';
```

A view can join tables

```
CREATE VIEW StaffWithBranchCity AS
  SELECT s.staffNo, s.name, s.position, b.city
  FROM Staff s JOIN Branch b ON s.branchNo =
  b.branchNo;
```

...or aggregate

```
CREATE VIEW BranchSalaryTotals AS
  SELECT branchNo, COUNT(*) AS staffCount,
  SUM(salary) AS totalSalary
  FROM Staff GROUP BY branchNo;
```

Nothing was copied — each is just a name bound to a SELECT, re-run against live data every time it's referenced.

Querying Views

A view is queried exactly like a base table

```
SELECT * FROM Branch003Staff  
WHERE salary > 10000  
ORDER BY salary DESC;
```

- This transparency is the whole point — application code doesn't need to know it's talking to a view
- You can join a view with a base table, or with another view
- The result is computed correctly each time by re-evaluating both underlying queries and combining them — the DBMS handles this transparently

Which Views Are Updatable

A view is generally updatable only if it:

- Is built from a SINGLE base table (no join)
- Does NOT use DISTINCT
- Does NOT use GROUP BY, HAVING, or an aggregate function
- Does NOT use set operations (UNION, INTERSECT, EXCEPT)
- Includes every column with no default value that disallows NULL

Updatable vs. Non-Updatable, in Practice

Branch003Staff — updatable

```
UPDATE Branch003Staff
SET salary = salary * 1.05
WHERE staffNo = 'SG37';
-- Legal: single-table, no-aggregate view
```

BranchSalaryTotals — not updatable

```
UPDATE BranchSalaryTotals
SET totalSalary = 50000
WHERE branchNo = 'B003';
-- ERROR: cannot update a view
-- that contains aggregate functions
```

- BranchSalaryTotals has no way to push “set totalSalary to 50000” back onto individual Staff rows — aggregation irreversibly threw away row-level detail
- StaffWithBranchCity (a join view) is likewise non-updatable in most systems — ambiguous which base table a changed column belongs to
- A handful of systems support INSTEAD OF triggers for custom logic — application-defined behavior layered on top, not automatic

Advantages and Disadvantages of Views

Advantages

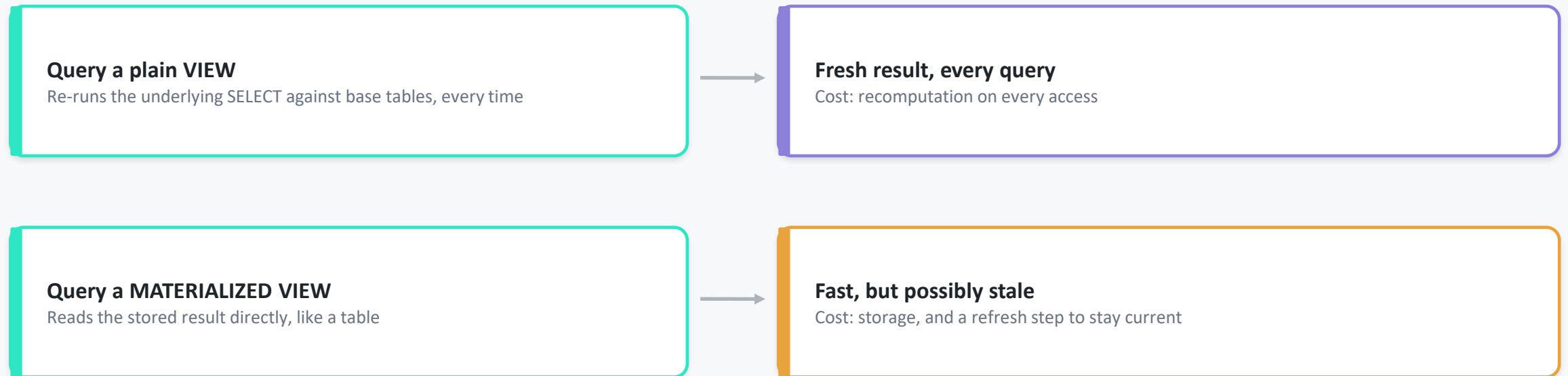
- Security — grant access to a view's rows/columns without exposing the whole base table
- Simplicity — hide complex joins/aggregations behind one simple name
- Logical data independence — the base schema can be restructured without breaking applications

Disadvantages

- Performance overhead — the underlying query re-runs on every reference
- Update restrictions — many useful views (joins, aggregates, DISTINCT) are simply not updatable

Ordinary View vs. Materialized View

An ordinary view stores no data and recomputes every time. A materialized view physically stores its result set on disk, refreshed periodically.



Creating Materialized Views

Only the keyword changes — the effect is very different

```
CREATE MATERIALIZED VIEW BranchSalaryTotals_MV AS
  SELECT branchNo, COUNT(*) AS staffCount,
         SUM(salary) AS totalSalary
  FROM Staff
  GROUP BY branchNo;
```

- BranchSalaryTotals_MV now occupies real storage, sized like the query's result set
- It does NOT automatically reflect a subsequent UPDATE to Staff until explicitly refreshed

Refreshing Materialized Views

```
REFRESH MATERIALIZED VIEW BranchSalaryTotals_MV;  
-- on-demand, blocks until complete  
  
REFRESH MATERIALIZED VIEW CONCURRENTLY  
  BranchSalaryTotals_MV;  
-- doesn't block concurrent reads while refreshing
```

Refresh strategy	When it runs	Best for
On-demand	Manually, via REFRESH	Reports run occasionally, staleness OK between runs
Scheduled	On a timer (nightly, hourly)	Dashboards with a known, bounded staleness window
On-commit	Automatically, every committing txn	Data that must never be stale (slower writes)

Advantages and Disadvantages of Materialized Views

Advantages

- Query performance — expensive joins/aggregations computed once at refresh time, not on every read
- Reduces load on base tables for read-heavy reporting and analytics workloads

Disadvantages

- Staleness — data can lag between refreshes; unacceptable for some apps (real-time balances)
- Storage cost — the result set is physically duplicated
- Refresh overhead — full, non-incremental refreshes can be expensive

Choosing Between a View and a Materialized View

Default to an ordinary view.

Reach for a materialized view specifically when profiling shows a query (usually a heavy aggregation or multi-way join) is run often enough, relative to how often its data changes, that recomputing it every time is measurably wasteful — and the application can tolerate the resulting staleness window.

If it can't tolerate any staleness at all, REFRESH ON COMMIT narrows the window to zero — at the cost of slower writes. No free lunch: only a trade-off you pick deliberately.

Key Takeaways

- A VIEW is a virtual table: a stored, named query with no data of its own, re-evaluated against live base-table data every time it's referenced.
- Views are queried like base tables, but only a subset — no join, DISTINCT, GROUP BY, or aggregate — are updatable.
- Views buy security, simplicity, and logical data independence, at the cost of recomputation overhead and restricted updatability.
- A MATERIALIZED VIEW physically stores its result set, trading storage and staleness for query speed — the opposite trade-off profile.
- Refresh strategy (on-demand, scheduled, on-commit) is the single most important design decision for any materialized view.